



Object Oriented Behavioral Modeling (Dynamic UML Design)

Chapter 8

Interaction Diagrams

- ◇ UML Specifies a number of interaction diagrams to **model dynamic aspects** of the system
- ◇ Dynamic aspects of the system
 - ◇ Messages moving among objects/
classes
 - ◇ Flow of control among objects
 - ◇ Sequences of events

Dynamic Diagram Types

- ◇ **Interaction Diagrams** - Set of objects or roles and the messages that can be passed among them. The sub types are
 - ◇ **Sequence Diagrams** - emphasize time ordering
 - ◇ **Communication Diagrams or Collaboration diagram** - emphasize structural ordering
 - ◇ Sequence and communication diagrams are semantically similar, they represent the same information
 - ◇ We can turn communication diagram into sequence diagrams and vice versa
 - ◇ Communication diagram are very useful for visualizing relationship between objects that collaborate with each other to perform a specific task.
- ◇ Which is difficult to determine from sequence diagram

Dynamic Diagram Types

◆ State Diagrams

- ◆ State machine consisting of states, transitions, events and activities of an object

◆ Activity & Swimlane Diagrams

- ◆ Emphasize and show flow of control among objects

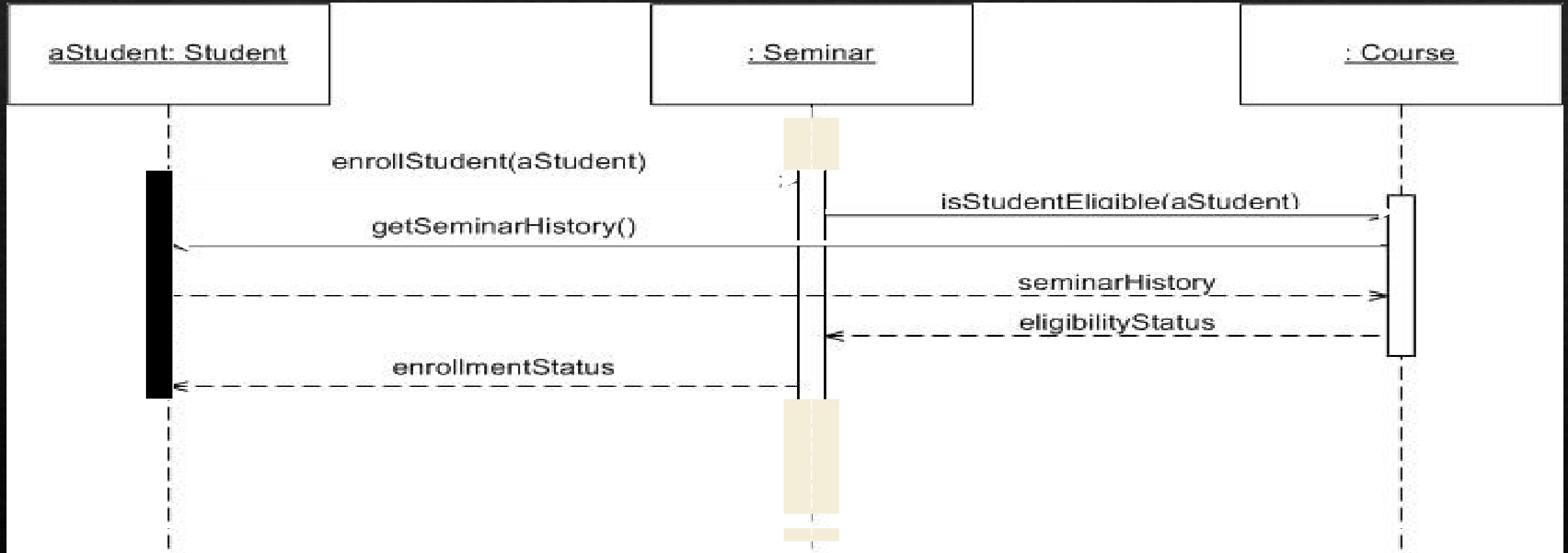
Sequence Diagram

Sequence Diagrams

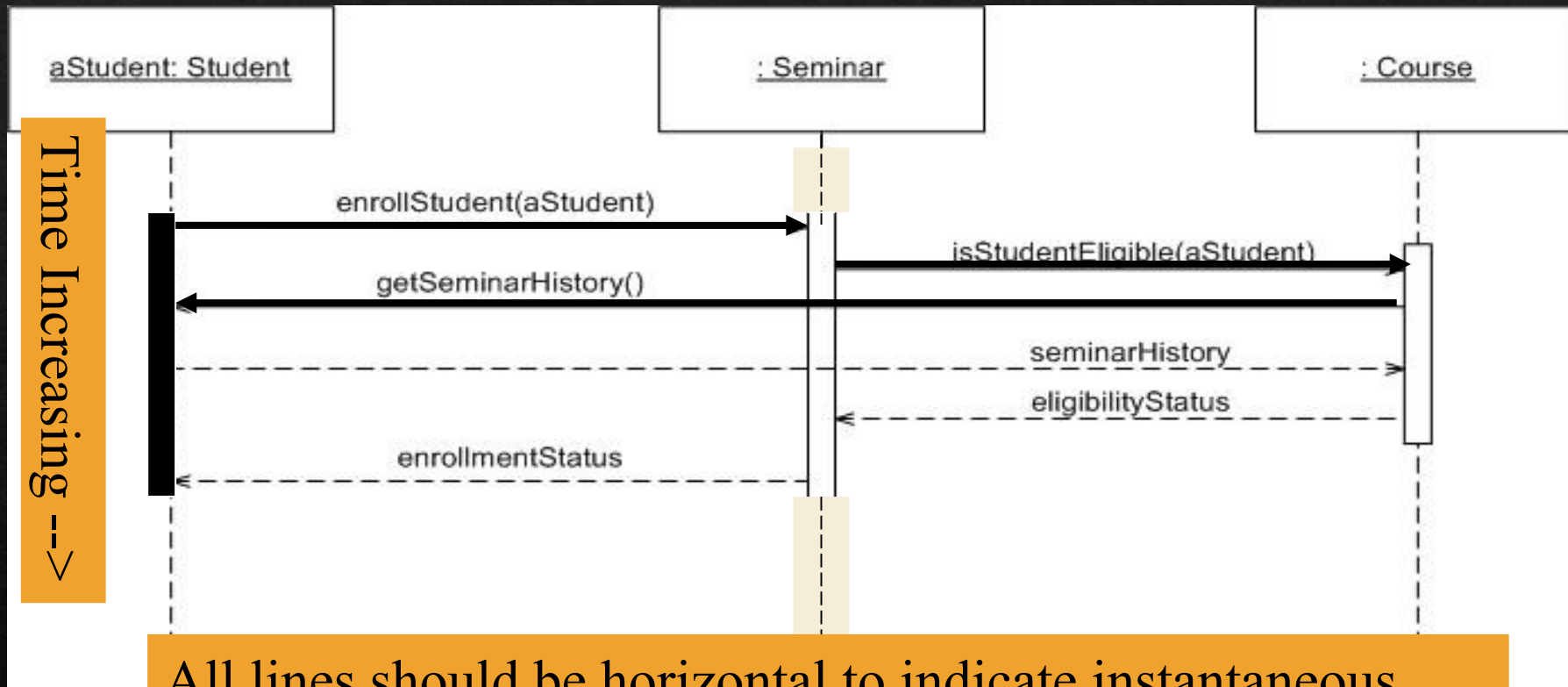
- Describe the flow of messages, events, actions between objects
- Show concurrent processes and activations
- Show time sequences that are not easily depicted in other diagrams
- Typically used during analysis and design to document and understand the logical flow of your system

Emphasis on time ordering!

Sequence Diagram



Sequence Diagram

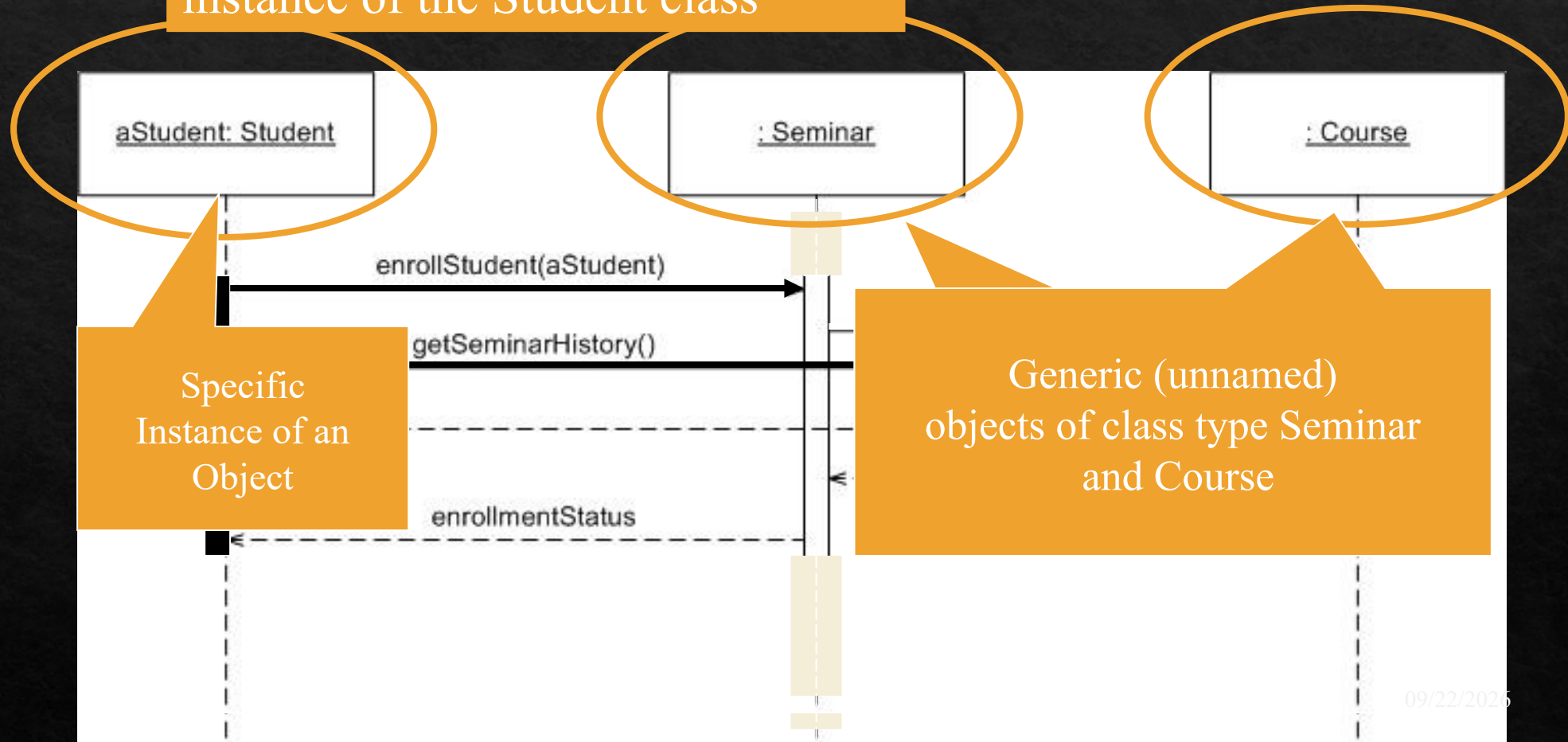


All lines should be horizontal to indicate instantaneous actions. Additionally if ActivityA happens before ActivityB, ActivityA must be above ActivityB

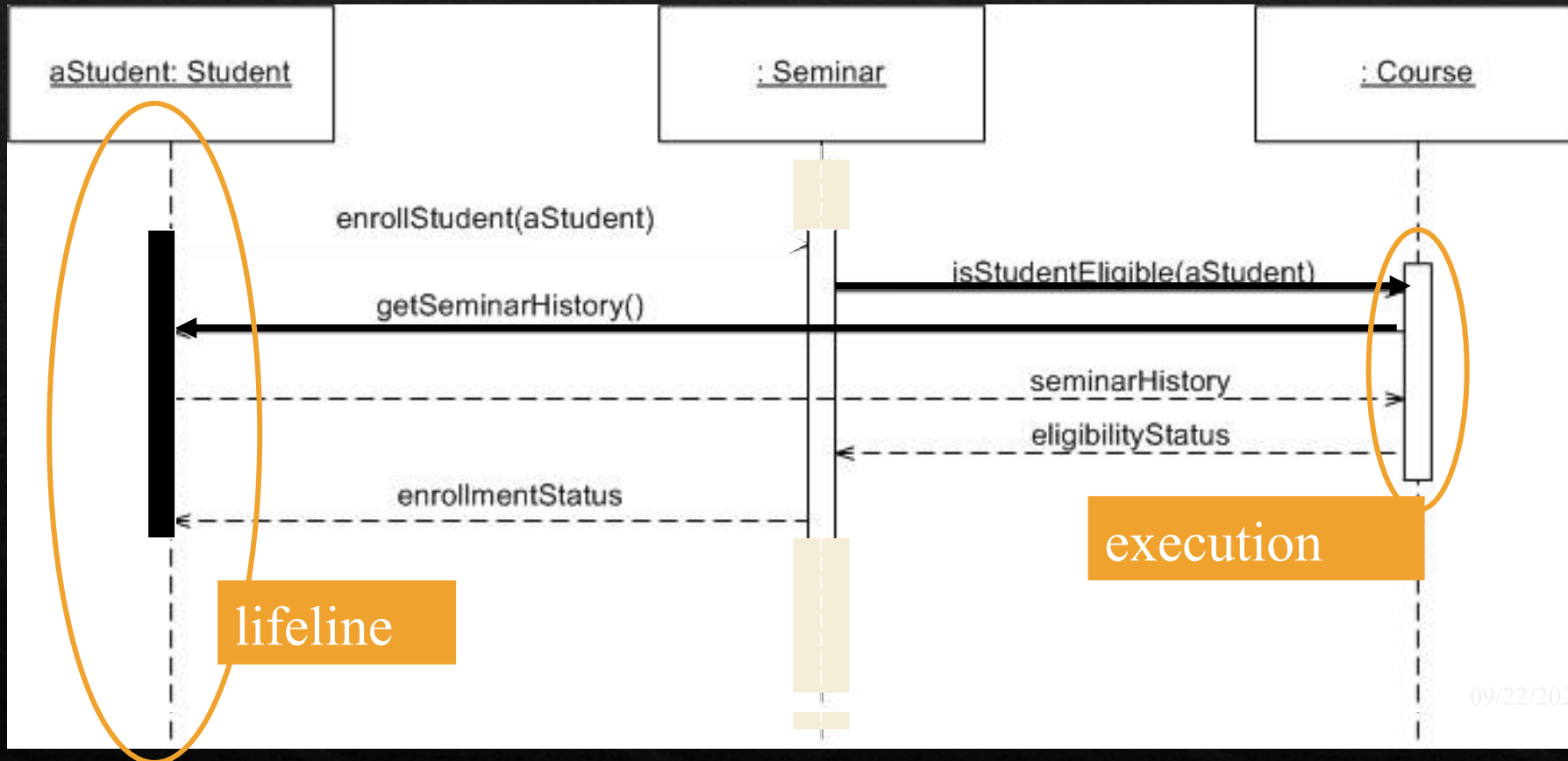
Lower = Later!

Sequence Diagram Components

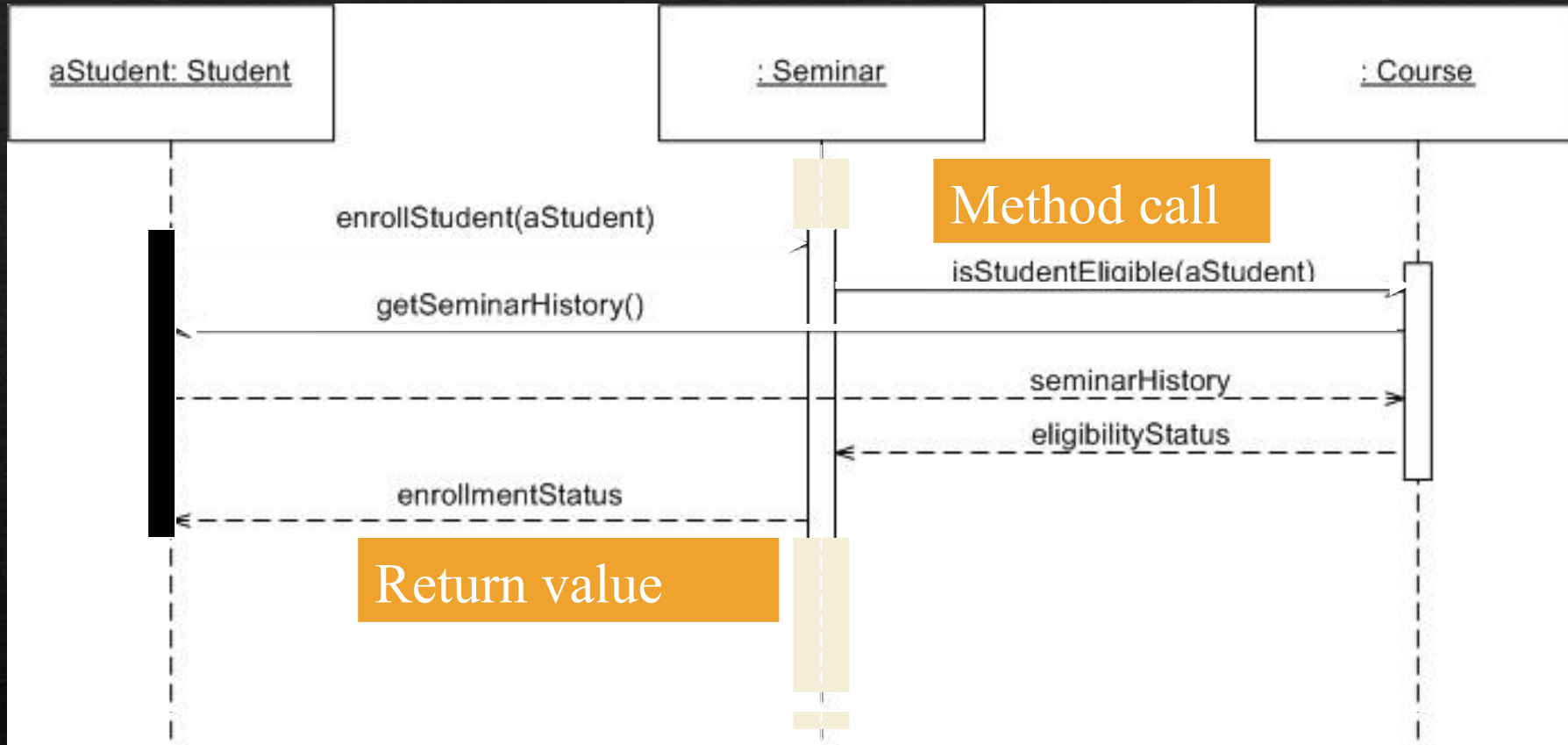
Objects: aStudent is a specific instance of the Student class



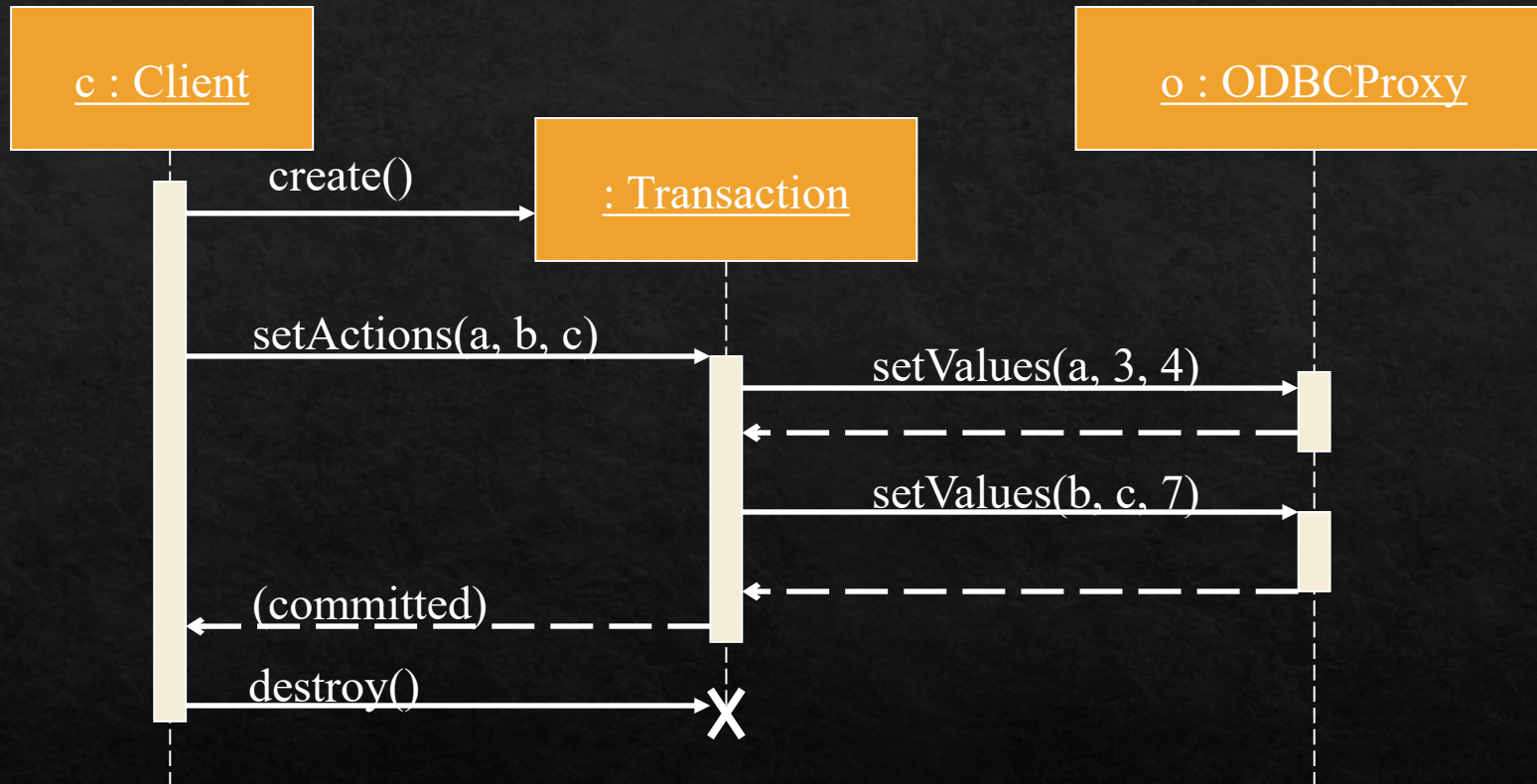
Sequence Diagram Components



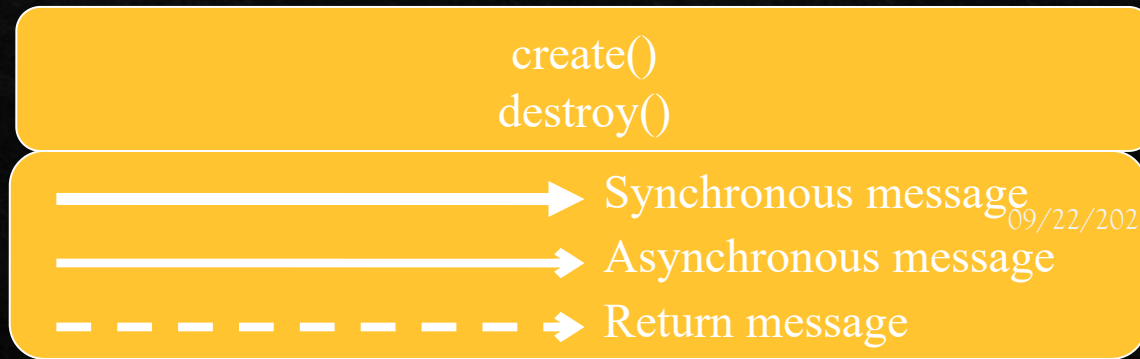
Sequence Diagram Components



Sequence Diagram Components



- A synchronous message requires a response before the interaction can continue.
- Asynchronous messages don't need a reply for interaction to continue.

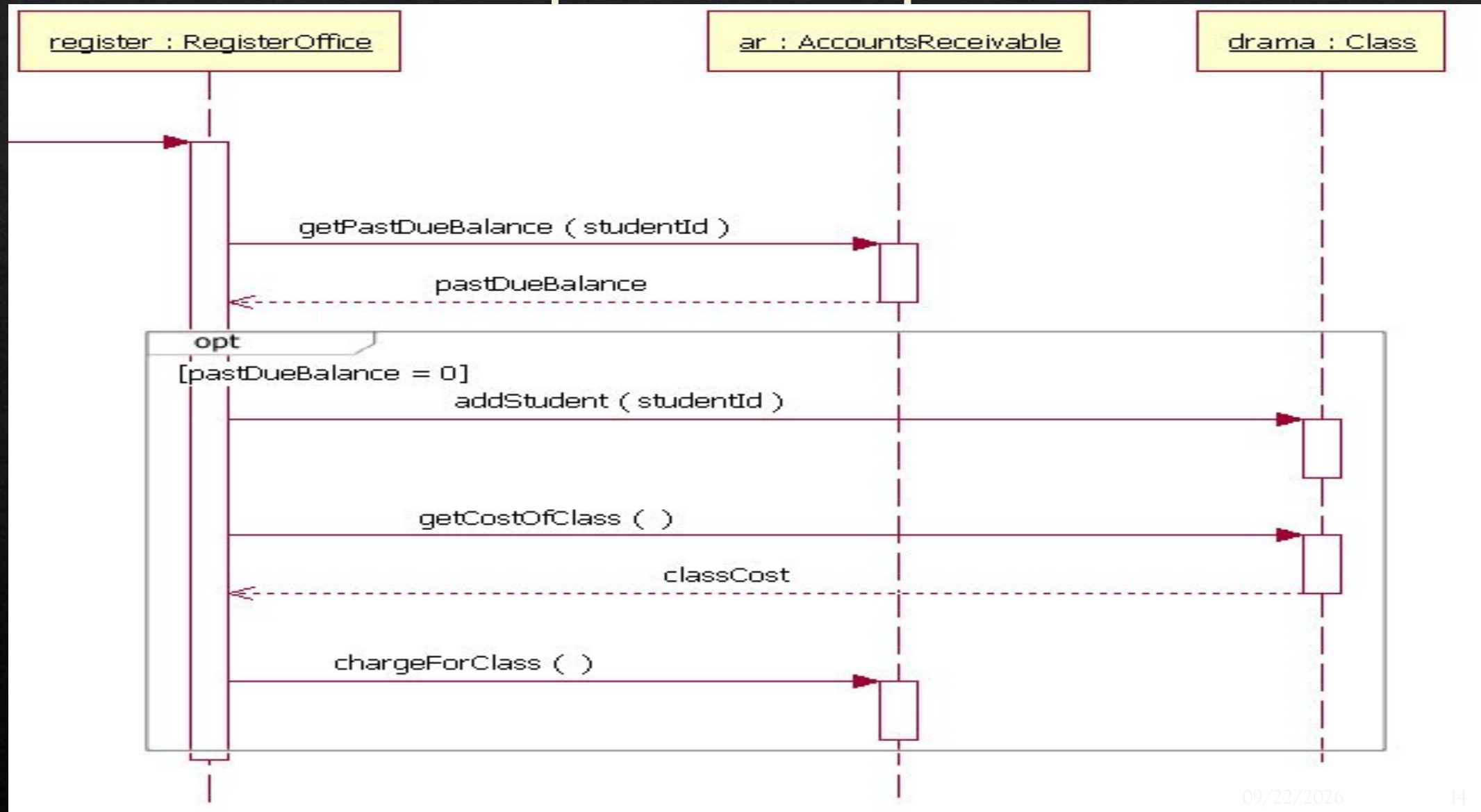


09/22/2026

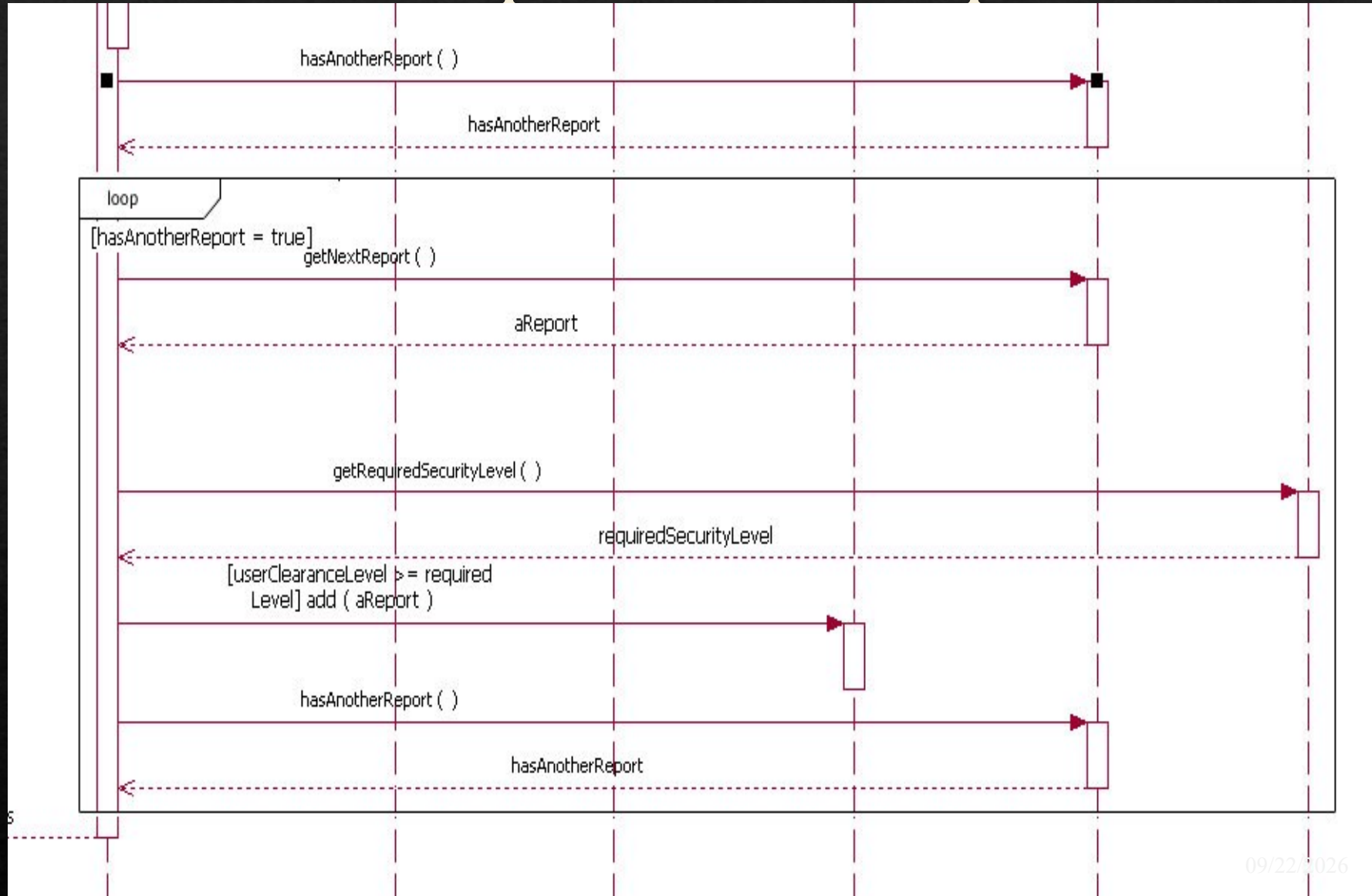
Components: alt/else



Components: option



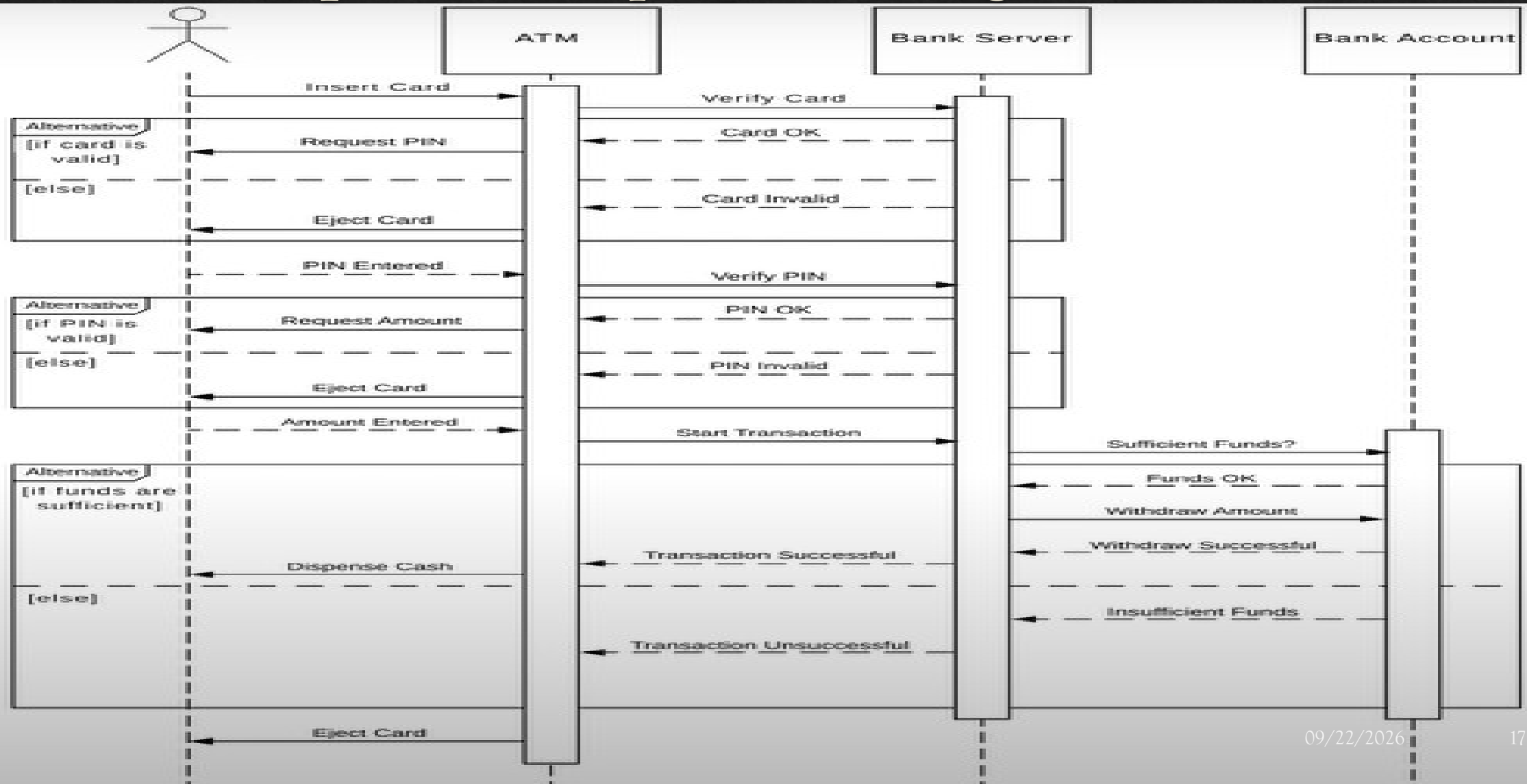
Components: loop



Rules of thumb

- ◆ Rarely use options, loops, alt/else
 - ◆ These constructs complicate a diagram and make them hard to read/interpret
 - ◆ Frequently it is better to create multiple simple diagrams
- ◆ Create sequence diagrams for use cases when it helps clarify and visualize a complex flow
- ◆ Remember: The goal of UML is communication and understanding

Example of Sequence Diagram for ATM



Collaboration or Communication Diagrams

What is a Collaboration Diagram

- ◆ Collaboration diagrams are extensions of object diagram which illustrate interactions between objects
- ◆ The collaboration diagram illustrate messages being sent between classes and objects (instances).
- ◆ Collaboration diagrams express *both the context of a group of object* (through objects and links) and the *interaction between these objects* (by representing message broadcasts)
- ◆ They provide an alternate view of the same information as sequence diagrams.
- ◆ In **sequence diagrams**, the focus is the **sequence of messages**; in **collaboration diagrams** the focus is the **structure of the message** that passes between the objects in the interaction.
- ◆ These diagrams illustrate the flow of messages between objects and the impl relationships between classes.

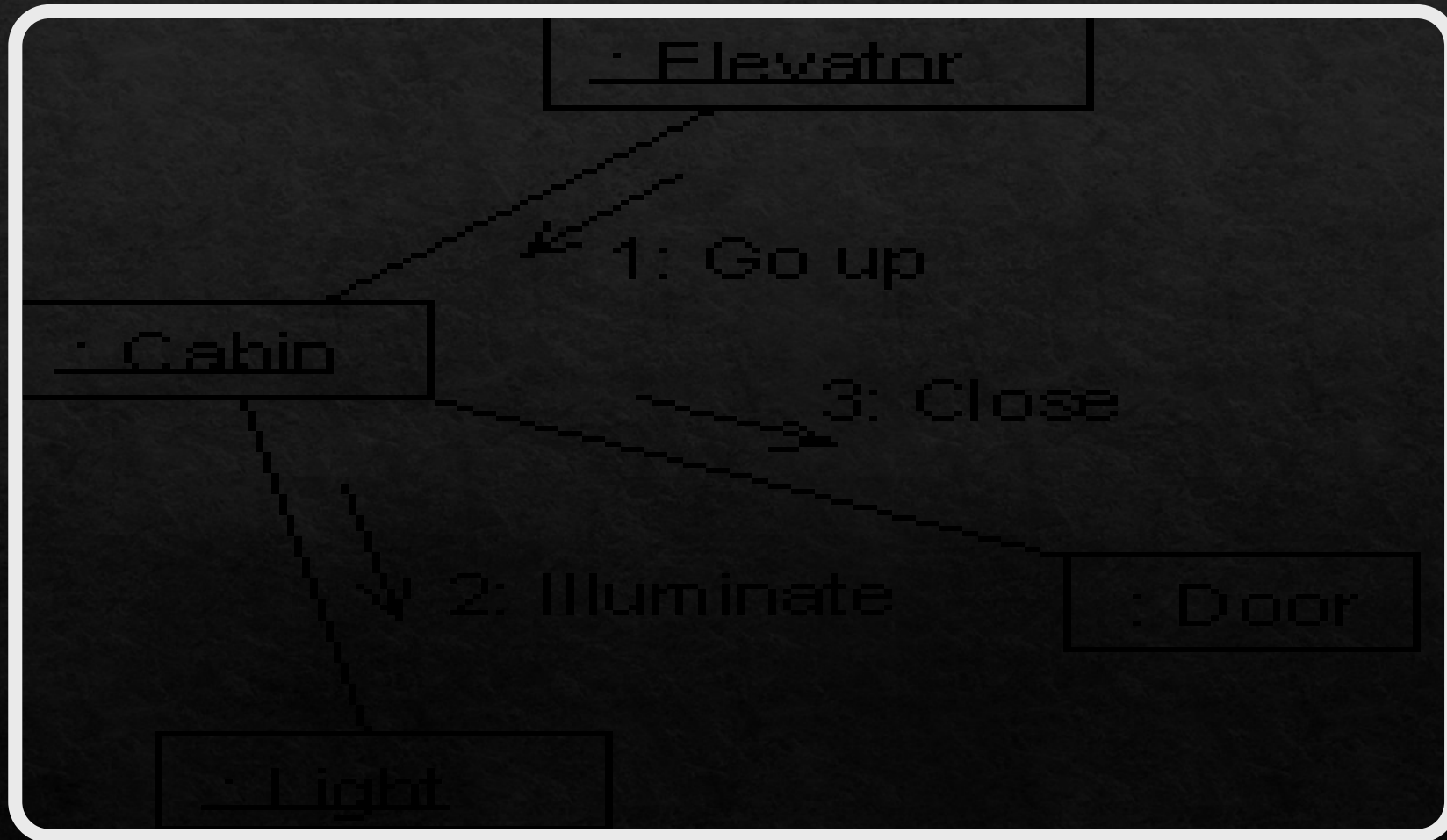
Purpose of the Collaboration Diagram

- ◆ They are very useful *visualizing the relationship between objects collaborating* to perform a particular task
- ◆ They provide a good view – although **static** - view of **interaction between objects** which may be **difficult to see at the class level**

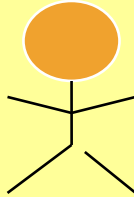
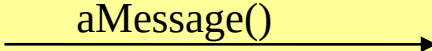
Collaboration Diagram

- ◆ Represents a Collaboration and Interaction
- ◆ Collaboration set of objects and their interactions in a speci context
- ◆ Interaction **set of messages exchange** in a collaboration produce a desired result

Ex. Collaboration Diagram



Collaboration Diagram Syntax

AN ACTOR	
AN OBJECT	
AN ASSOCIATION	
A MESSAGE	

Objects

- ◆ **Objects** rectangles containing the object signature :

- ◆ **object name : object Class**

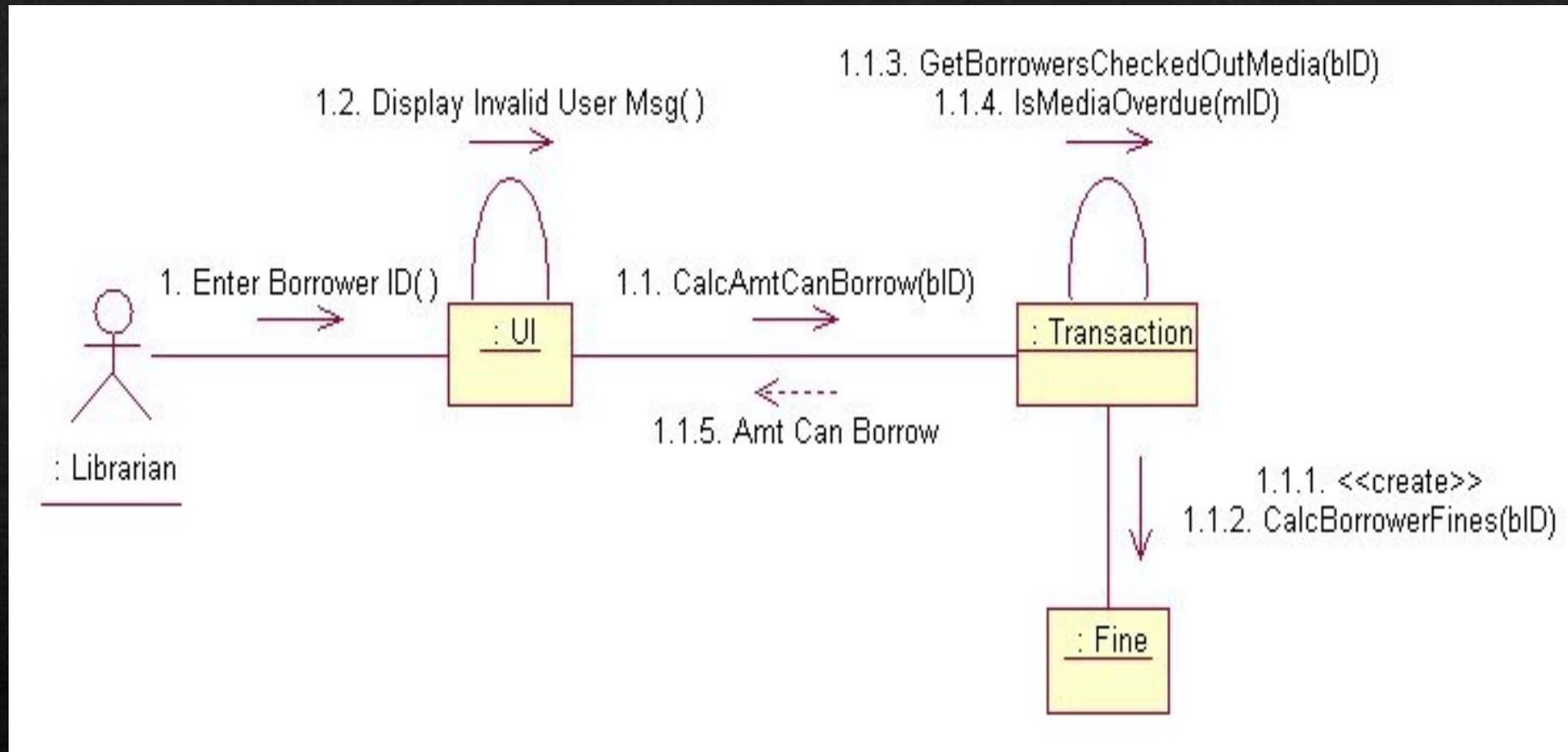
- ◆ object name (optional) - starts with lowercase letter

- ◆ class name (mandatory) - starts with uppercase letter

- ◆ Objects connected by lines actor can appear

Objects

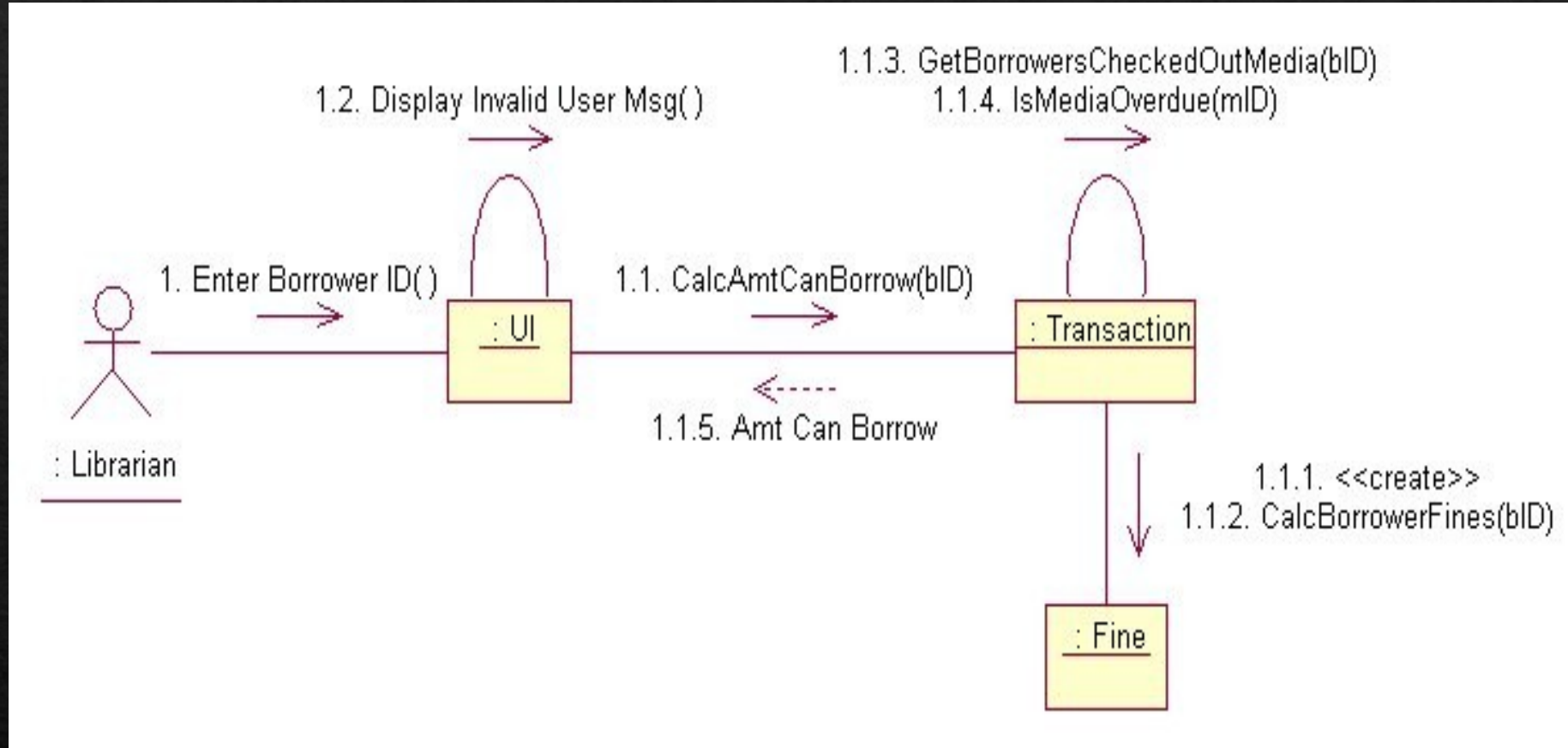
- ◆ Objects participating in a collaboration come in two flavors—supplier and client
- ◆ Supplier objects are the objects that supply the method that is being called and therefore **receive** the message
- ◆ Client objects call methods on supplier objects, and therefore **send** messages.



- transaction object acts as a supplier to the UI (user interface)
in turn, the fine object is a supplier to the transaction client object.

Links

- ◆ The connecting lines drawn between objects are links
- ◆ They enable you to see the relationships between objects
- ◆ This symbolizes the ability of objects to send messages to each other
- ◆ A **single link** can support **one or more messages** sent between objects



- The visual representation of a link is a straight line between two objects.
- *If an **object** sends messages to itself, the link carrying these messages is represented as a loop icon.*
- This loop can be seen on both the UI object and the Transaction object.

Messages

- ◆ An interaction is implemented by a group of objects that collaborate by exchanging messages
- ◆ Messages in collaboration diagrams are shown as arrows point from the Client object to the Supplier object.
- ◆ Typically, messages represent a client invoking an operation on a supplier object

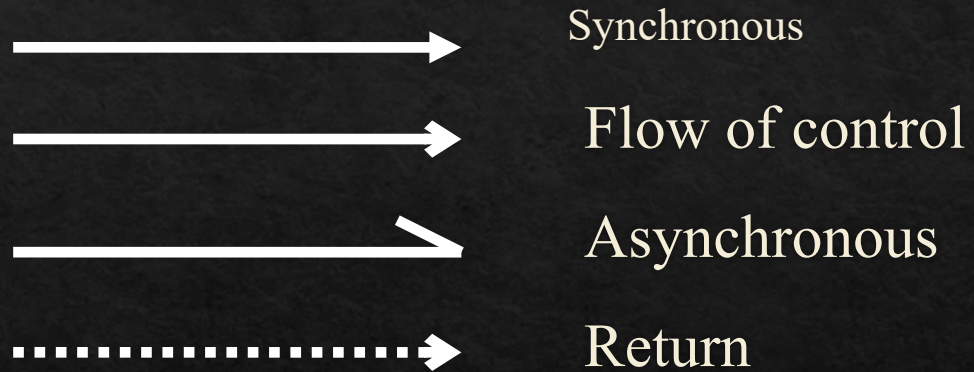
Messages

- ◆ Message icons have one or more messages associated with them
- ◆ Messages are composed of message text prefixed by a sequence number
- ◆ Time is not represented explicitly in a collaboration diagram, and result the various messages are numbered to indicate the sending order

Flow by Numbers

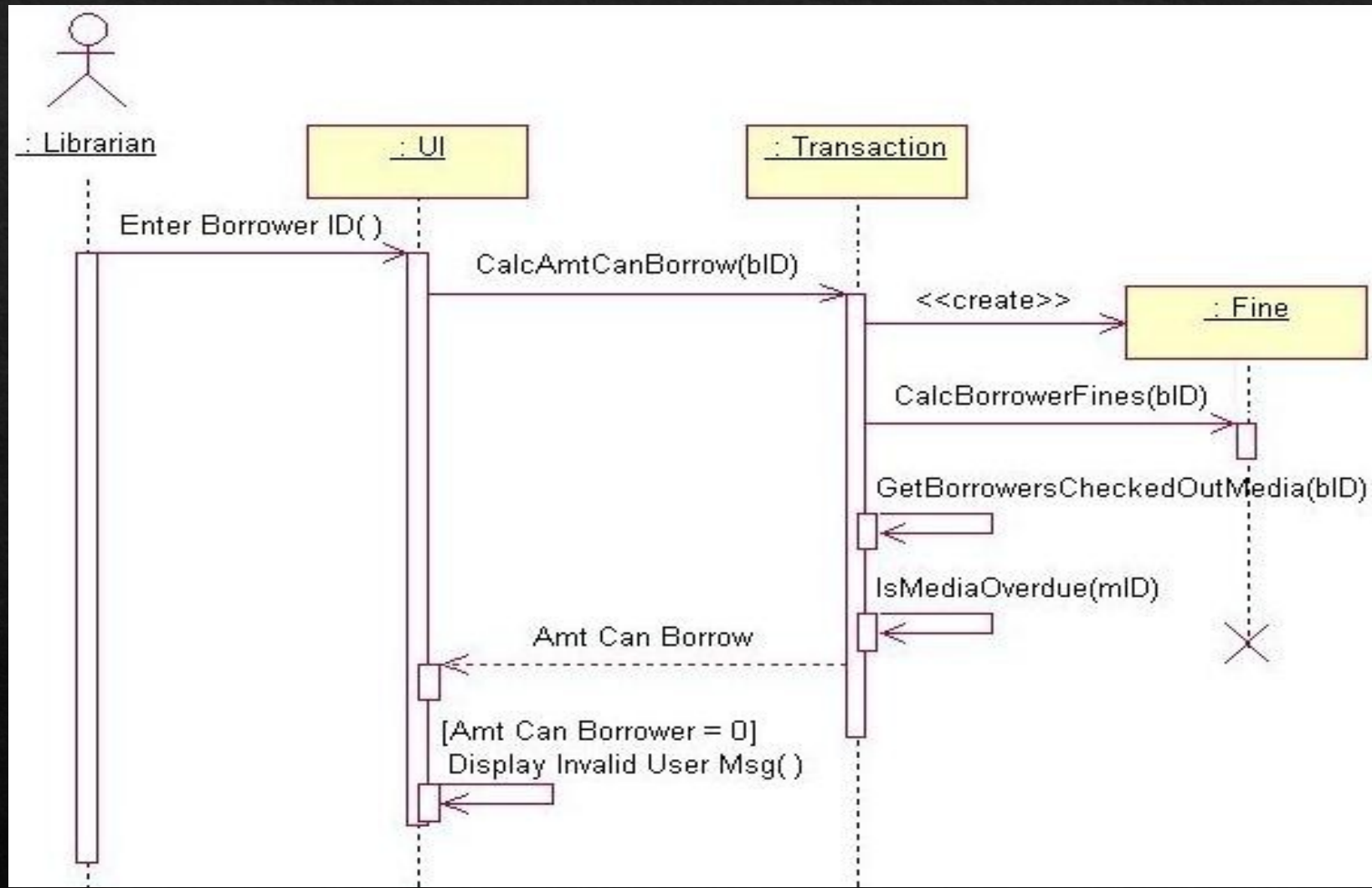
- 1. Enter Borrower ID
 - 1.1 CalcAmtCanBorrow
 - 1.1.1 <<create>>
 - 1.1.2 CalcBorrowerFines
 - 1.1.3 GetBorrowersCheckedOutMedia
 - 1.1.4 IsMediaOverdue
 - 1.1.5 Amt Can Borrow
 - 1.2 Display Invalid User Msg

Message Flow Notation

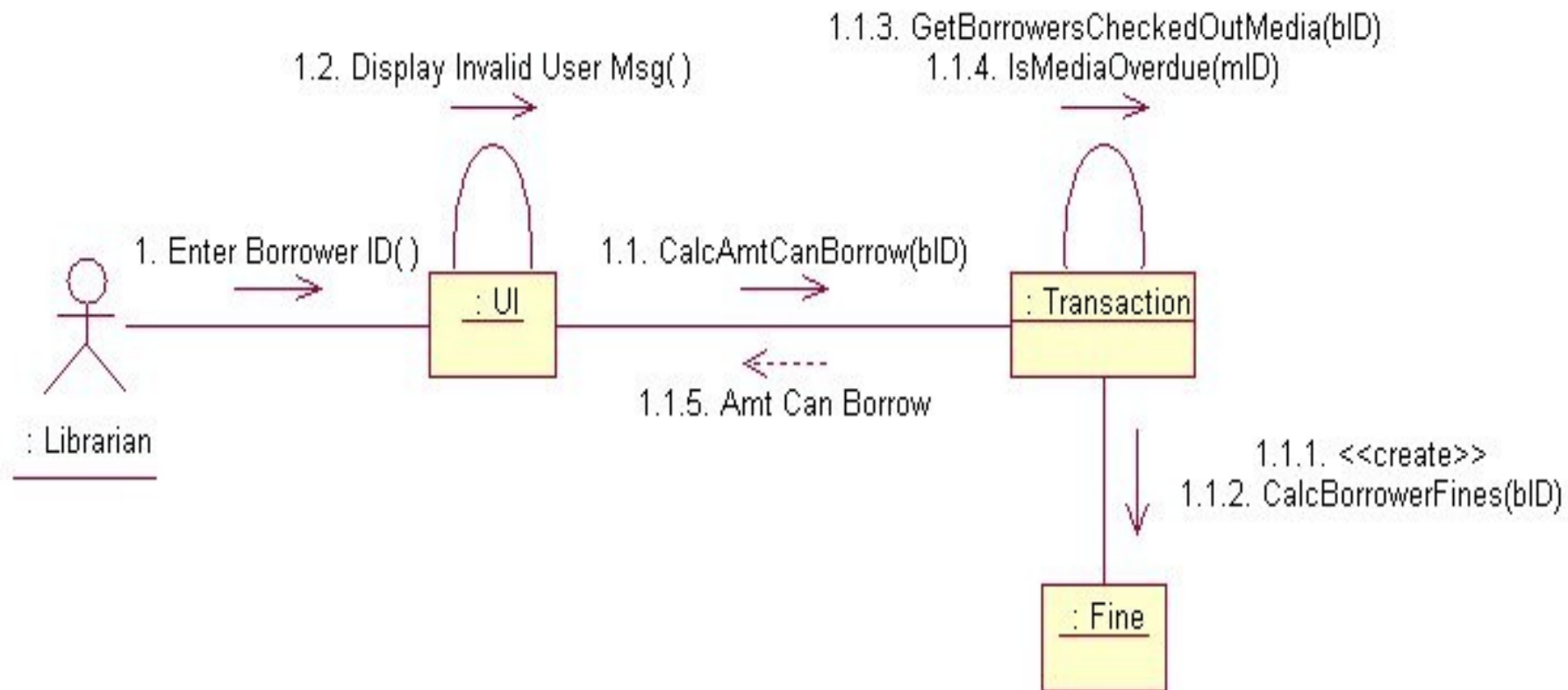


Collaboration vs Sequence Diagram

- ◆ In reality, sequence diagrams and collaboration diagrams show the **same information**, but just present it differently
- ◆ Not used as often as sequence diagrams but are closely related



Sequence diagram is better at 'time ordering'



Collaboration diagram is better at showing the relationship between objects

Number of Messages

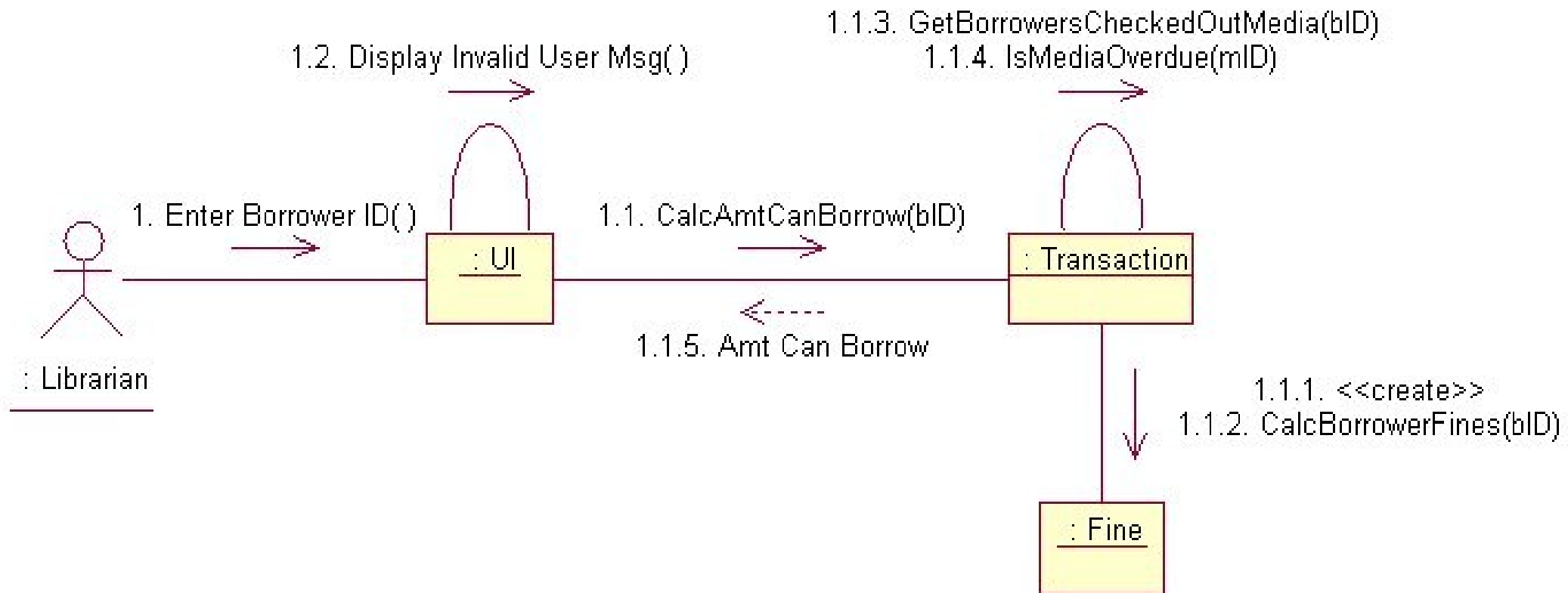
- ◆ In sequence diagrams, each message icon represents a single message.
- ◆ In collaboration diagrams, a message icon can represent one or more messages.
- ◆ Notice between the Transaction and Fine objects - there is a single message icon, but there are two messages (1.1.1 and 1.1.2) associated with the icon.

Steps to Creating a Collaboration Diagram

1. Determine the scope of the diagram- the use case it relates to
2. Place the objects that participate in the collaboration on the diagram
 - ◆ Remember to place the most important objects towards the center of the diagram.
3. If a particular object has a property or maintains a state that is important to the collaboration, set the initial value of the property or state

Steps to Creating a Collaboration Diagram

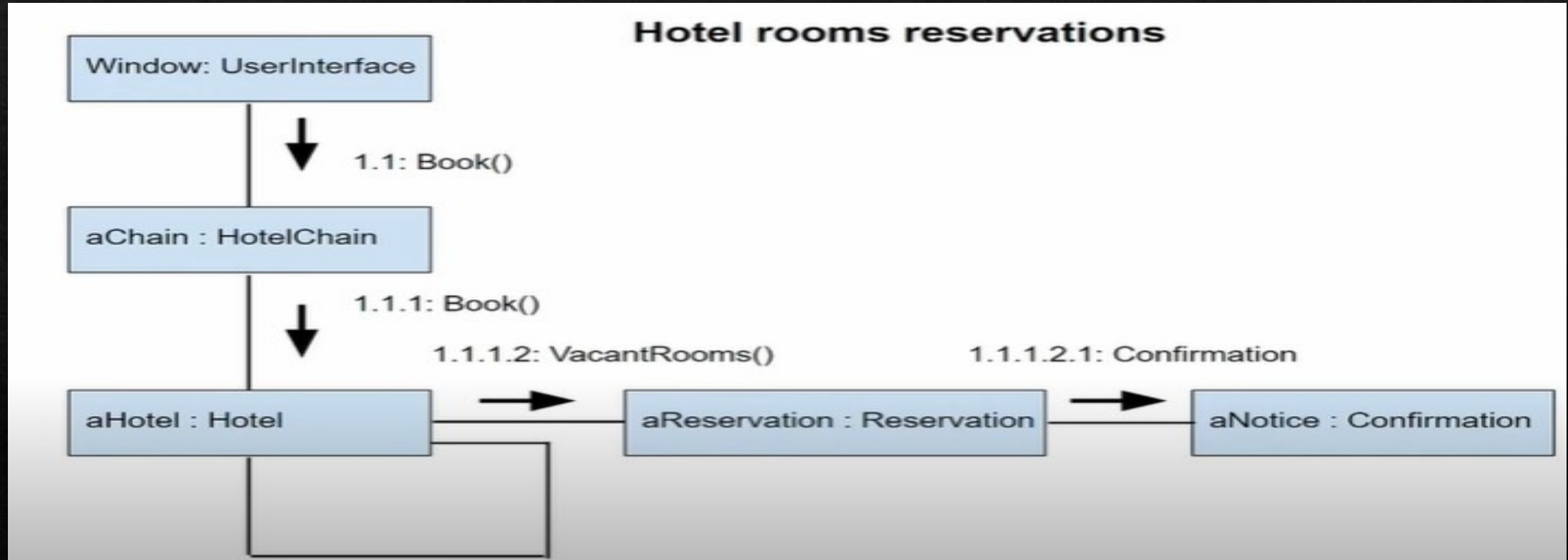
4. Create links between the objects
4. Create messages associated with each link
4. Add sequence numbers to each message corresponding to the time-ordering of messages in the collaboration



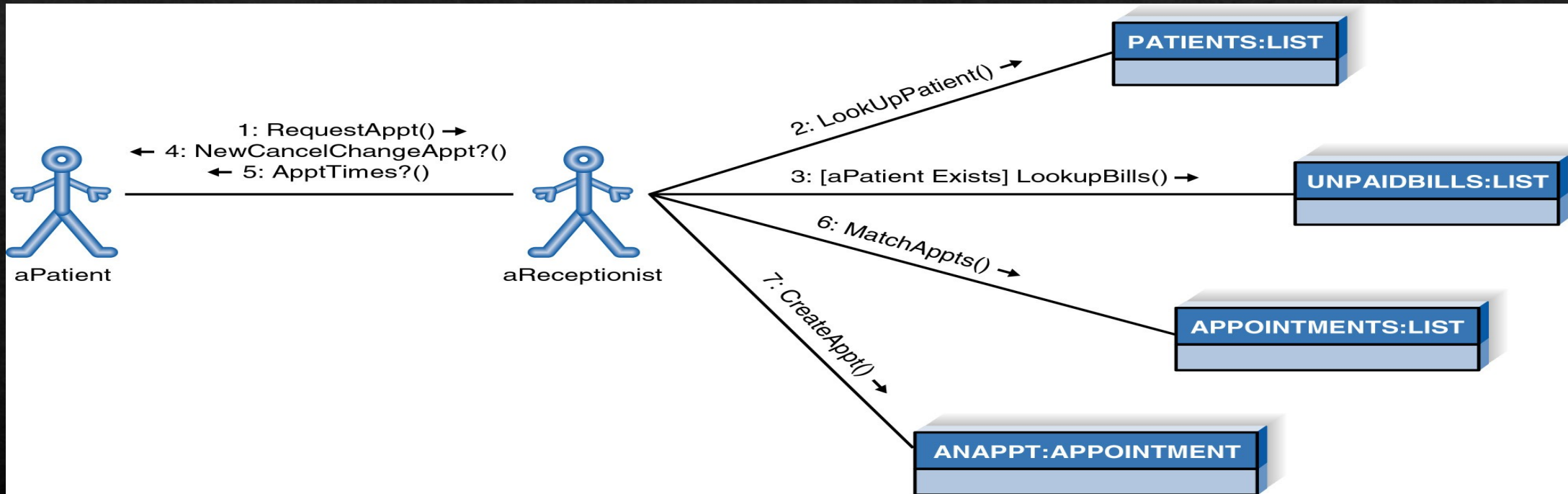
Change State of an Object



Example Collaboration Diagram



Example Collaboration Diagram



Dennis: SAD
Fig: 8-6 W-33 100% of size
Fine Line Illustrations (516) 501-0400

State Machine Diagram

What is a State Machine Diagram?

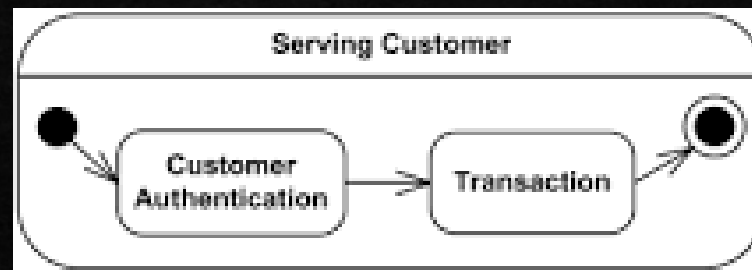
- ◆ State machine diagrams are used to **capture the behavior of a software system.**
- ◆ UML State machine diagrams can be used to model the *behavior of a class, a subsystem, a package, or even an entire system.*
- ◆ It is also called a State chart or State Transition diagram.

What is a State Machine Diagram?

- ◆ A state machine is any device that **stores the status of an object at a given time and can change status or cause other actions based on the input it receives.**
- ◆ States refer to the different combinations of information that an object can hold, not **how the object behaves.**

State diagram symbols and components

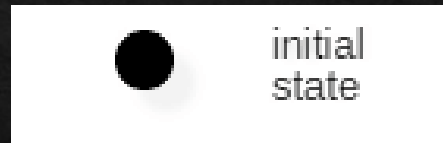
- ◆ This list summarizes the most common shapes you may encounter.
- ◆ **Composite state**
 - ◆ Generally, **composite state** is defined as **state** that has substates (nested **states**). Substates could be sequential (disjoint) or concurrent (orthogonal).



State diagram symbols and components...

◆ **Initial state**

- ◆ The initial state symbol is used to indicate the beginning of a state machine diagram.



◆ **Final state**

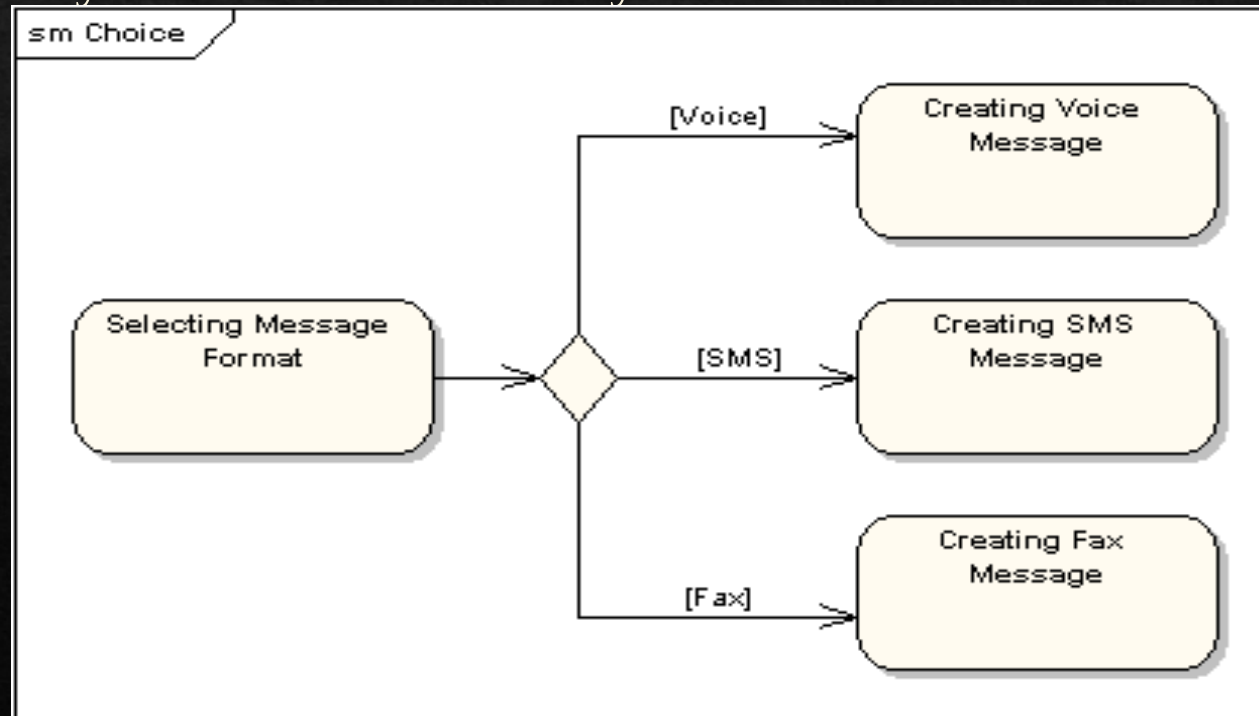
- ◆ This symbol is used to indicate the end of a state machine diagram.



State diagram symbols and components...

◇ Choice pseudo state

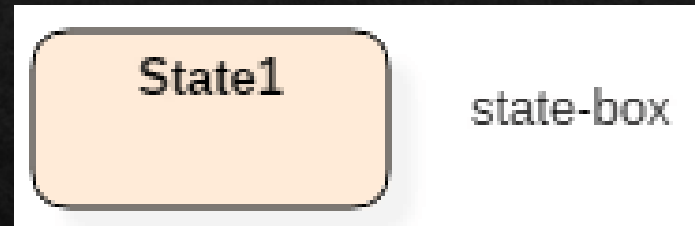
- ◇ A diamond symbol that indicates a dynamic condition with branched potential results.



State diagram symbols and components...

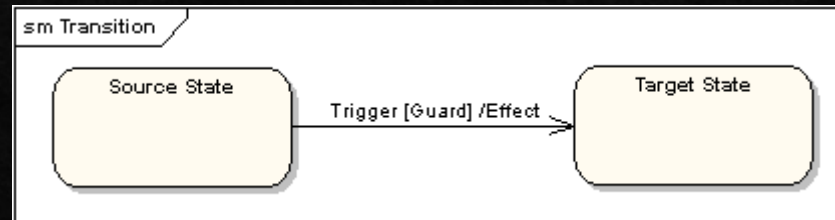
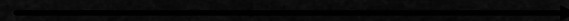
◆ State

- ◆ A rectangle with rounded corners that indicates the current nature of an object.



◆ Transition

- ◆ An arrow running from one state to another that indicates a changing state.

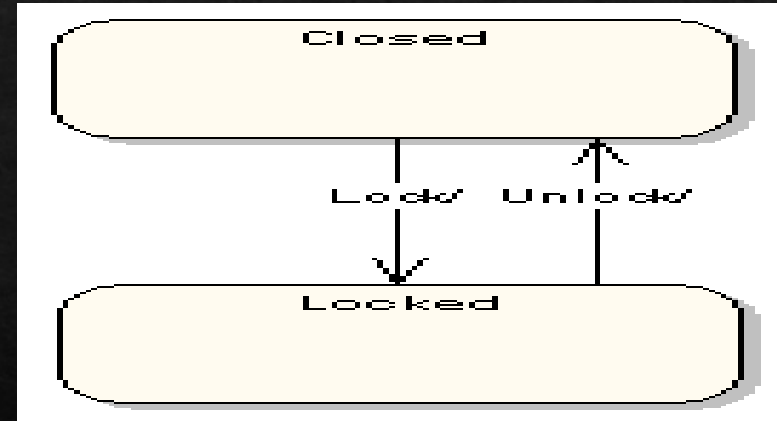


State diagram symbols and components...

◆ Trigger

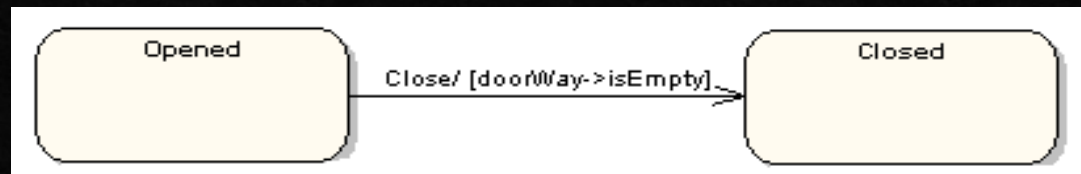
- ◆ A type of message that actively moves an object from state to state, written above the transition arrow.

◆ Event



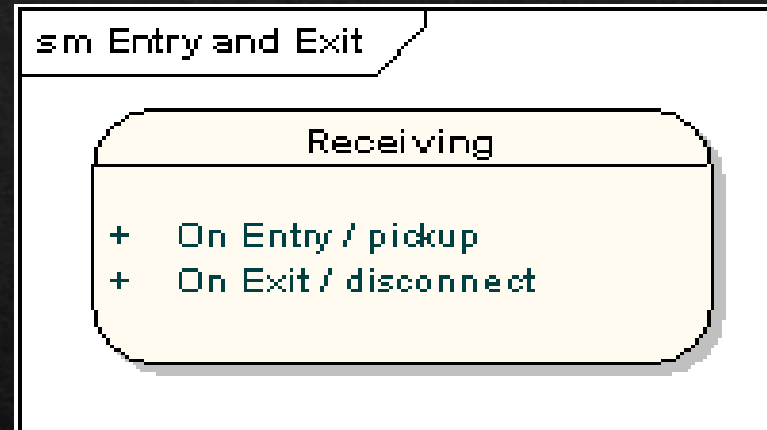
◆ Guard

- ◆ A Boolean condition that allows or stops a transition, written above the transition arrow.



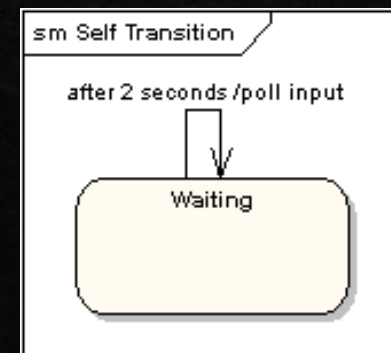
State diagram symbols and components...

◆ State Actions



◆ Self-Transitions

- ◆ A state can have a transition that returns to itself, as in the following diagram. This is most useful when an effect is associated with the transition.

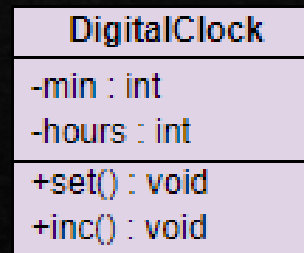


Example

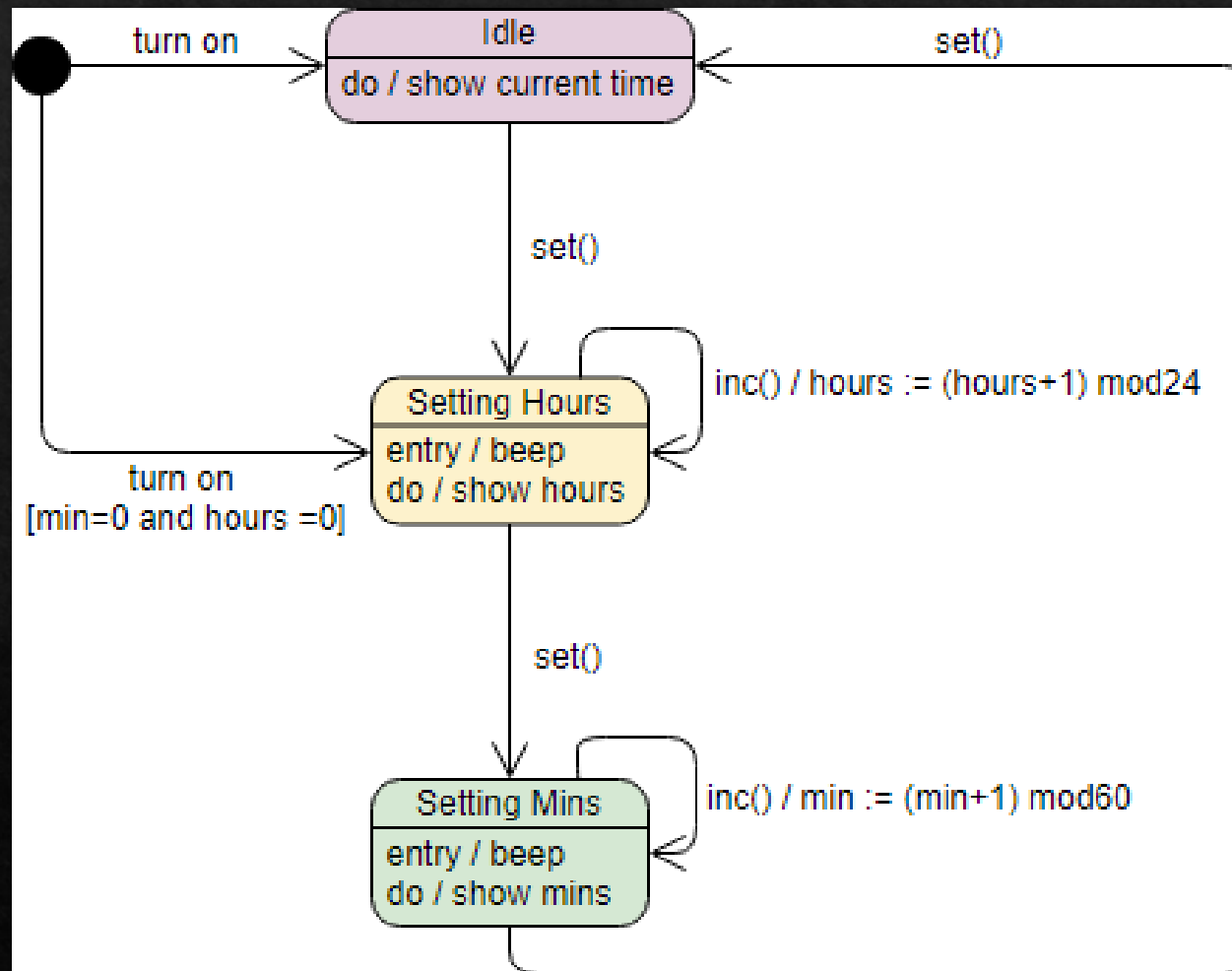
- ◆ The Digital Clock State Machine diagram example below shows the interface of a simple digital clock:



- ◆ The state machine diagram where the class it is attached:



The state Diagram for modeling the behavior of the Digital Clock.



How to draw a State chart diagram?

- ◇ To draw a state diagram, one must identify all the possible states of any particular entity.
 1. **Identify entities that have complex behavior** or identify a class participating in behavior whose lifecycle is to be specified
 2. Model states – **Determine the initial and final states of the entity**
 3. Model transitions
 4. Model events – **Identify the events that affect the entity**
 5. Working from the initial state, trace the impact of events and identify intermediate states
 6. Identify any **entry and exit actions on the states**
 7. Expand states using substates where necessary
 8. If the entity is a class, check that the action in the state are supported by the operations and relationships of the class, and if not extend the class
 9. Refine and elaborate as required

Rules

- ◆ Following rules must be considered while drawing a state chart diagram:
 1. The name of a state transaction must be unique.
 2. The name of a state must be easily understandable and describe the behavior of a state.
 3. If there are multiple objects, then only essential objects should be implemented.
 4. Proper names for each transition and an event must be given.

Activity Diagram

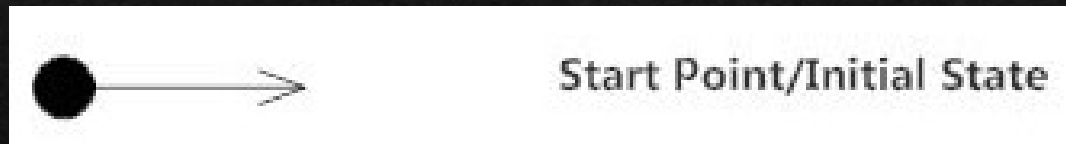
What is an Activity Diagram?

- ◆ An activity diagram visually presents a series of actions or flow of control in a system similar to a flowchart or a data flow diagram.
- ◆ Activity diagrams are often used in **business process modeling**.
- ◆ They can also describe the steps in a use case diagram.
- ◆ Activities modeled can be sequential and concurrent.
- ◆ In both cases an activity diagram will have a beginning (an initial state) and an end (a final state).

Basic Activity Diagram Notations and Symbols

◆ Initial State or Start Point

- ◆ A small filled circle followed by an arrow represents the initial action state or the start point for any activity diagram. For activity diagram using swimlanes, make sure the start point is placed in the top left corner of the first column.



- **Final State or End Point**

- An arrow pointing to a filled circle nested inside another circle represents the final action state.



Basic Activity Diagram Notations and Symbols...

◆ Activity or Action State

- ◆ An action state represents the non-interruptible action of objects.



◆ Action Flow

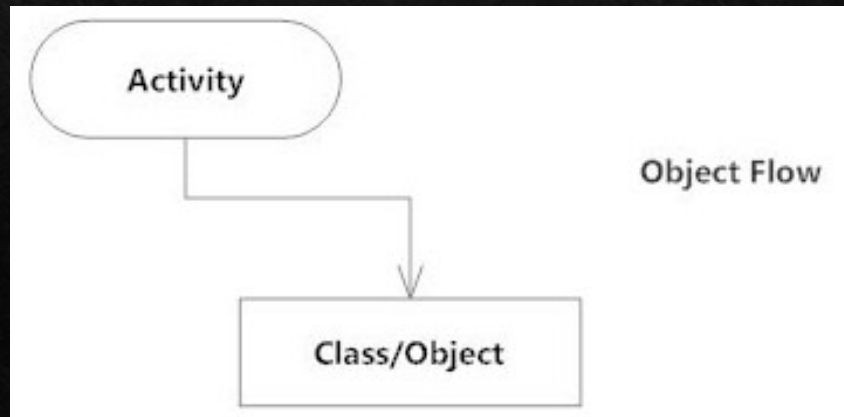
- ◆ Action flows, also called edges and paths, illustrate the transitions from one action state to another. They are usually drawn with an arrowed line.



Basic Activity Diagram Notations and Symbols...

◇ Object Flow

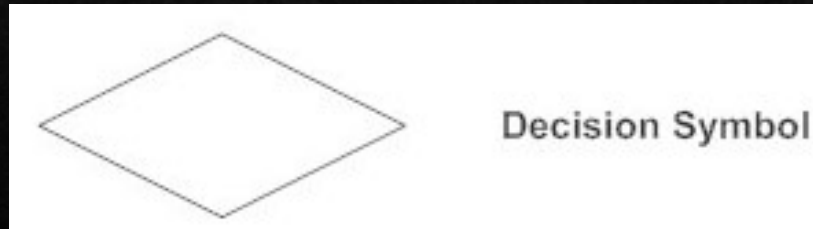
- ◇ Object flow refers to the creation and modification of objects by activities.
- ◇ An **object flow arrow from an action to an object** means that the *action creates or influences the object*.
- ◇ An **object flow arrow from an object to an action** indicates that *the action state uses the object*.



Basic Activity Diagram Notations and Symbols...

◆ Decisions and Branching

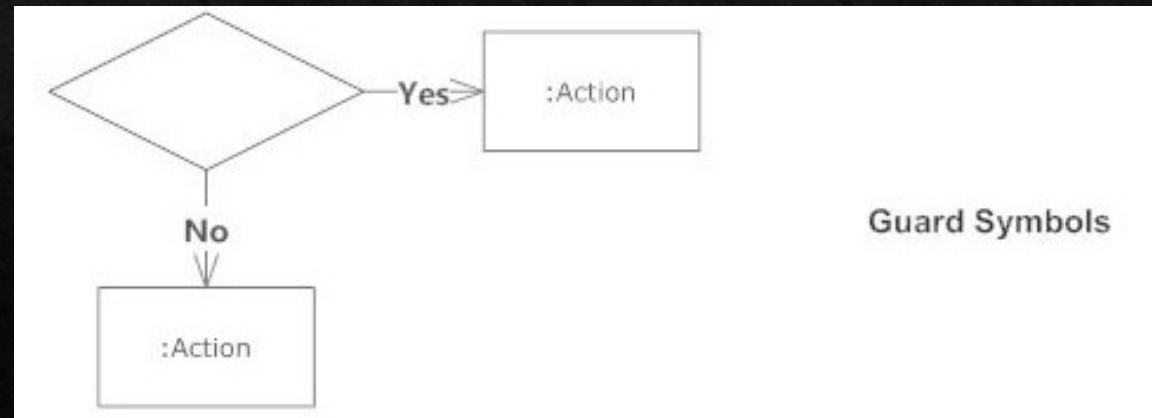
- ◆ A **diamond** represents a decision with alternate paths.
- ◆ When an activity requires a decision prior to moving on to the next activity, add a diamond between the two activities.
- ◆ The outgoing alternates should be labeled with a condition or guard expression.
- ◆ You can also label one of the paths "else."



Basic Activity Diagram Notations and Symbols...

◆ Guards

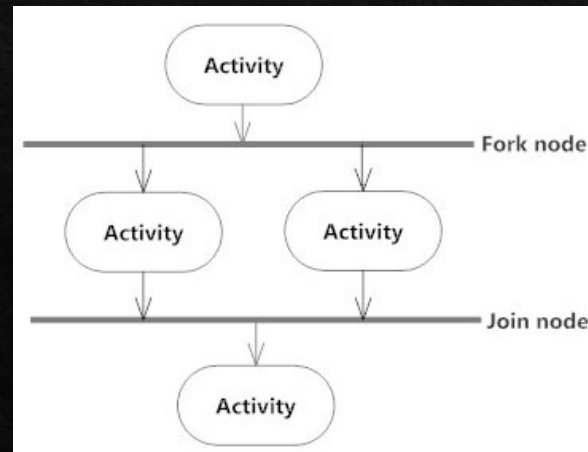
- ◆ In UML, guards are a **statement written** next to a decision diamond that must be true before moving next to the next activity.
- ◆ These are not essential, but are useful when a specific answer, such as "Yes, three labels are printed," is needed before moving forward.



Basic Activity Diagram Notations and Symbols...

◇ Synchronization

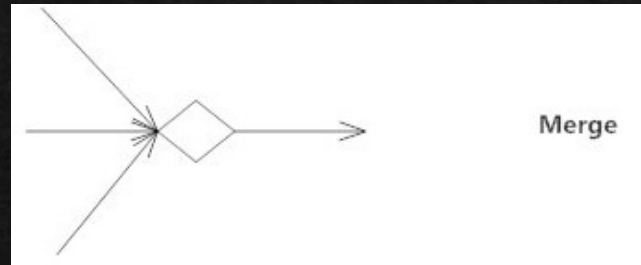
- ◇ **A fork node:** is used to split a **single** incoming flow **into multiple concurrent flows**.
- ◇ It is represented as a straight, slightly thicker line in an activity diagram.
- ◇ **A join node:** joins **multiple** concurrent flows back **into a single** outgoing flow.
- ◇ A fork and join node used together are often referred to as synchronization.



Basic Activity Diagram Notations and Symbols...

◆ Merge Event

- ◆ A merge event brings together multiple flows that are not concurrent.



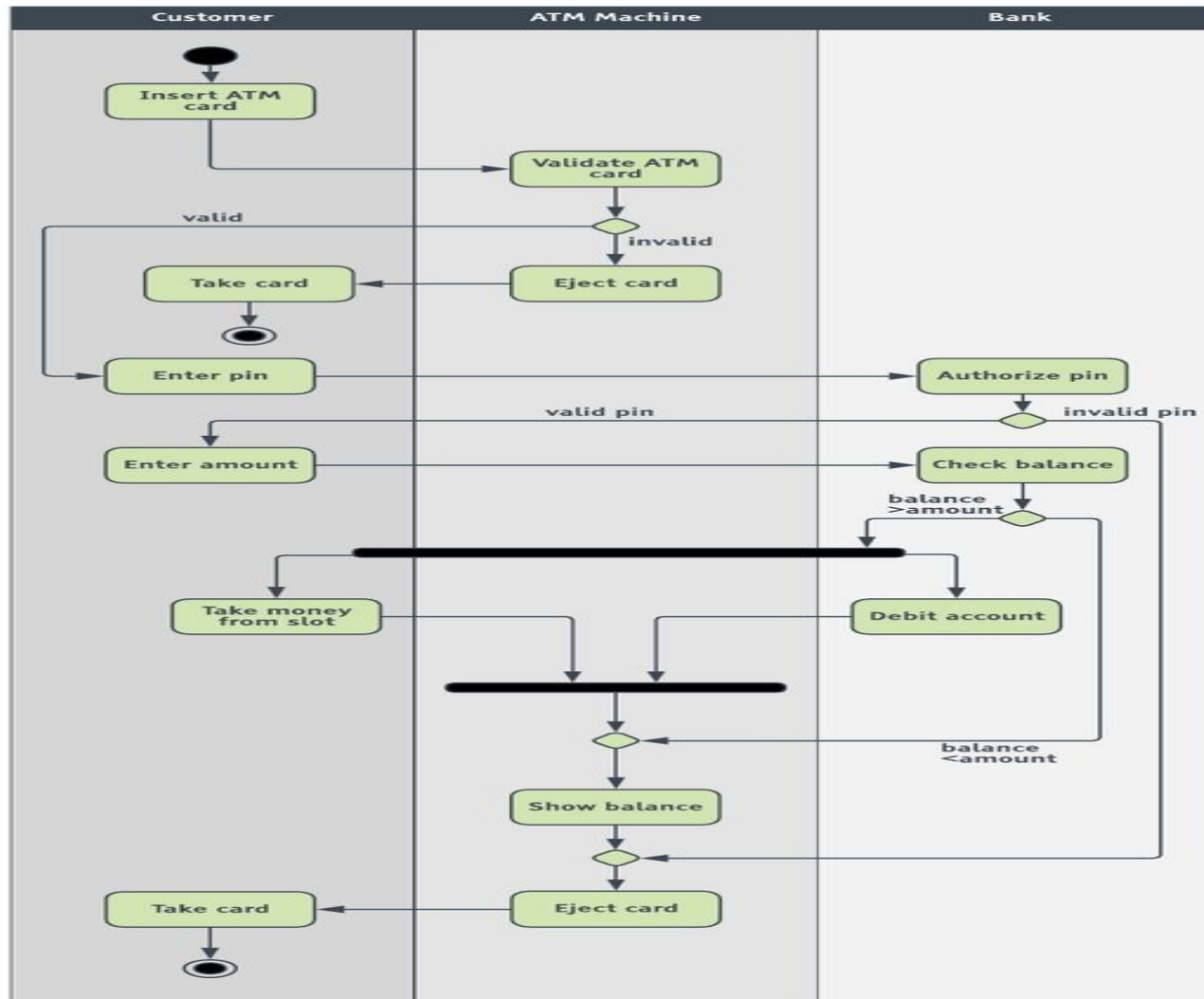
◆ Swim lanes

- ◆ A swim lane is a way to group activities performed by the same actor on an activity diagram or to group activities in a single thread.

Steps to model Activity Diagram

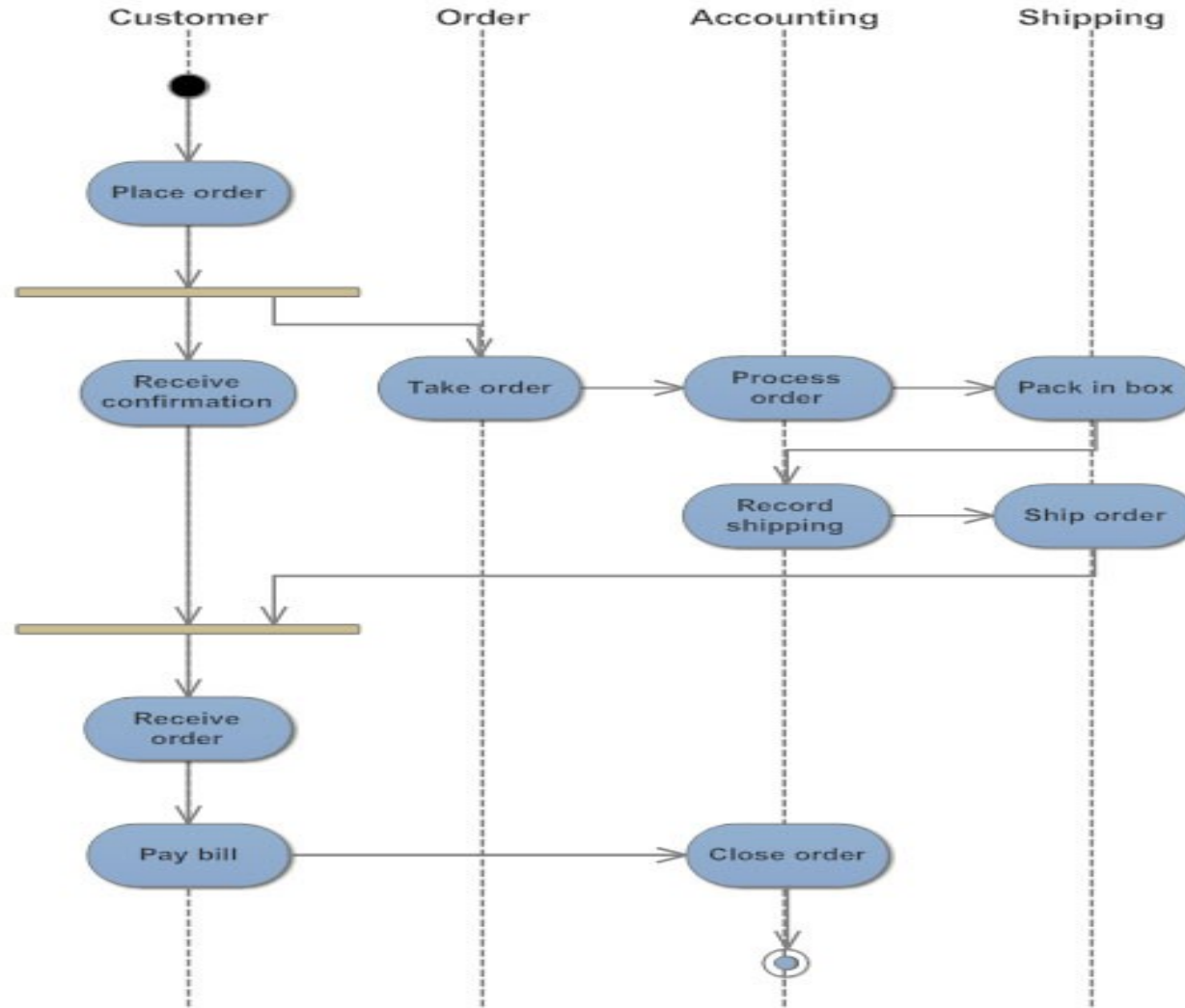
1. Identify candidate use cases, through the examination of business workflows
2. Identify pre- and post-conditions (the context) for use cases
3. Model workflows between/within use cases
4. Model complex workflows in operations on objects
5. Model in detail complex activities in a high level activity Diagram

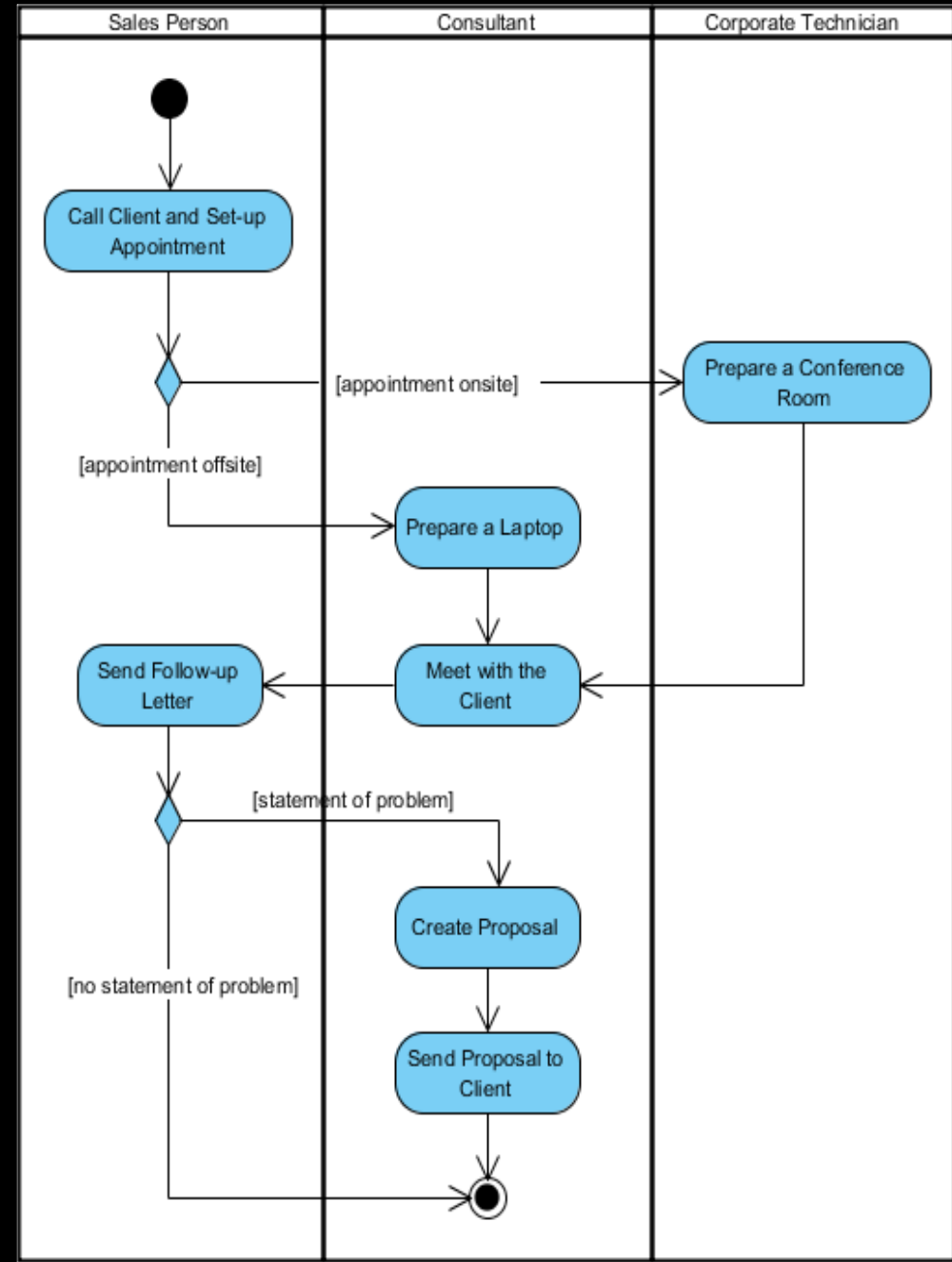
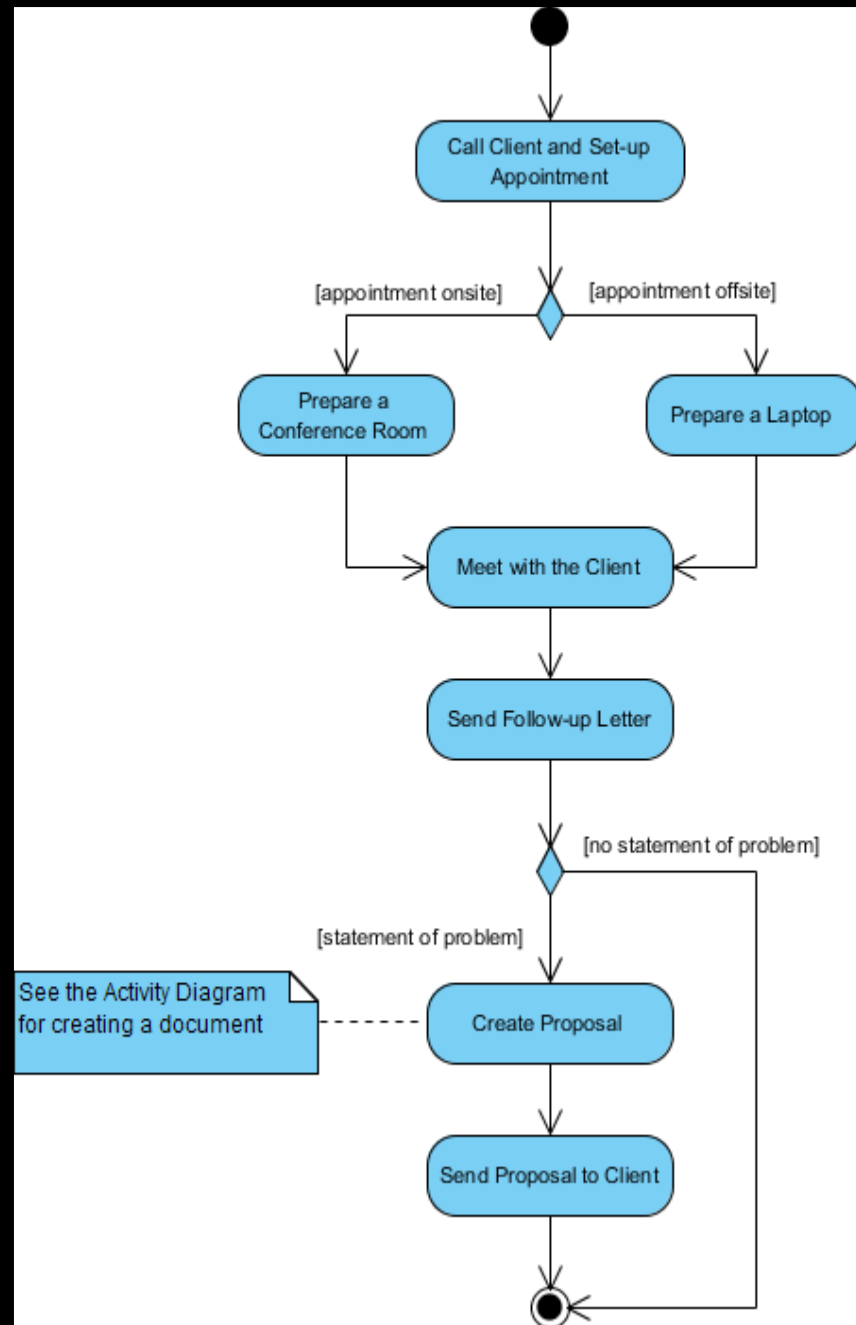
ATM Machine Activity diagram



Another Example

UML Activity Diagram: Order Processing





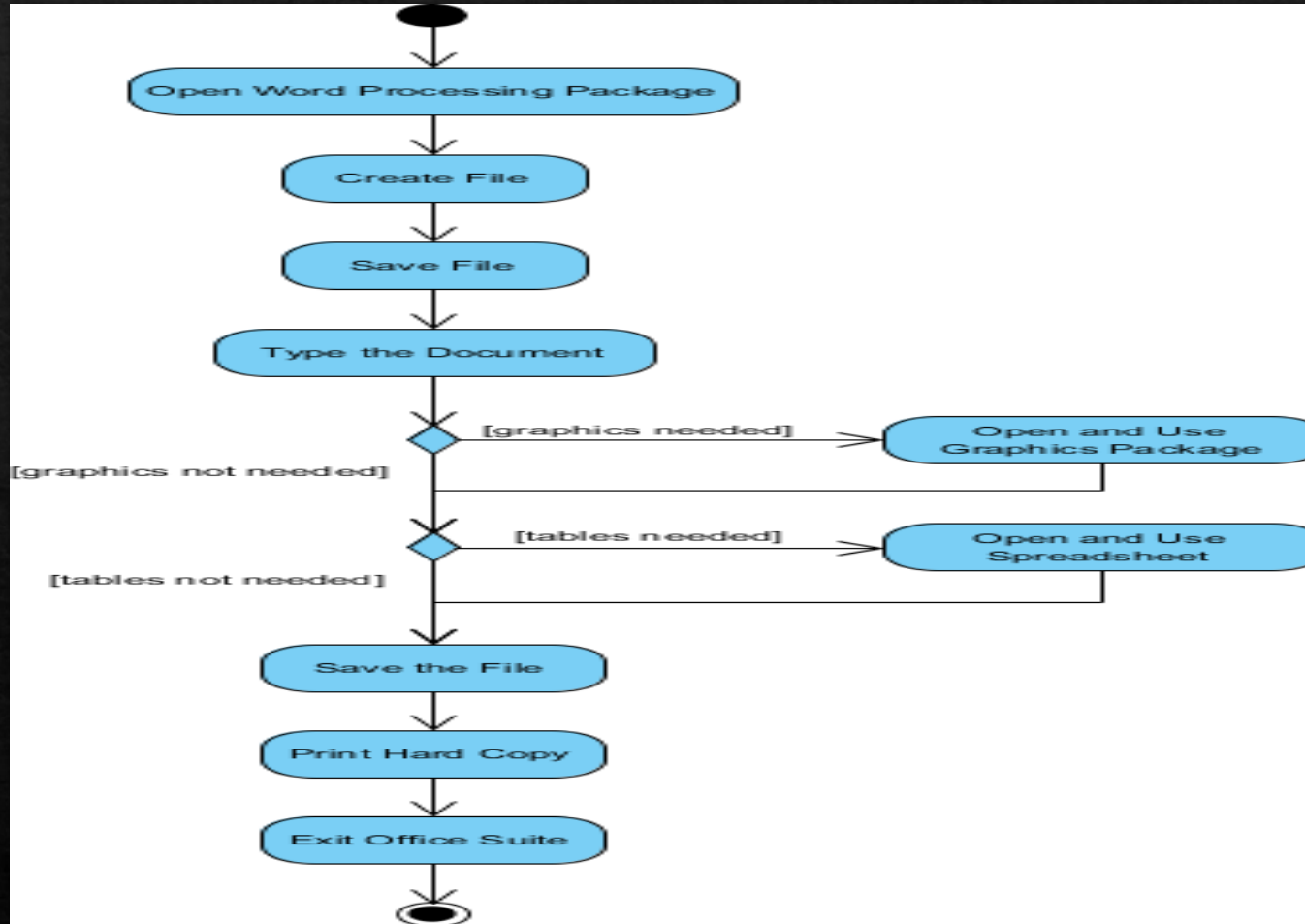
Exercise

Activity Diagram - Modeling a Word Processor

The activity diagram example below describes the workflow for a word process to create a document through the following steps:

1. Open the word processing package.
2. Create a file.
3. Save the file under a unique name within its directory.
4. Type the document.
5. If graphics are necessary, open the graphics package, create the graphics, and paste the graphics into the document.
6. If a spreadsheet is necessary, open the spreadsheet package, create the spreadsheet, and paste the spreadsheet into the document.
7. Save the file.
8. Print a hard copy of the document.

Possible answer for the previous exerci



END of the Course